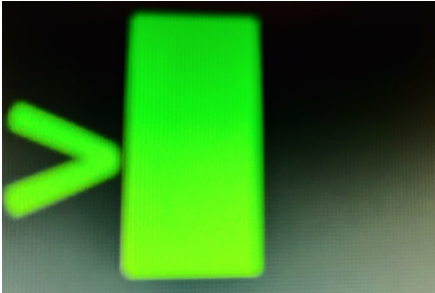


Linux Command Line Introduction

How come we have a command line? Isn't that like, uh, so 1970s?



The terminal can be a bit daunting at first, however there really isn't anything to be scared of. It's easy with a little bit of patience, and it's the true way to master an operating system.

Just as cavemen learned to develop their skills from grunting and groaning to writing meaningful representations on the walls of caves, The computer world started off with the keyboard and the [CLI \(Command Line Interface\)](#). No mice, no graphical user interface (GUI), no icons. All text.

A brief history of the command line

Xerox's PARC research center managed to invent the very first graphical user interface in the early 1980s, however, When Linux first came to be in 1991, most things were still text. True, Windows 3.0 was out, and MacOS had been with us since 1984 but the PC world was still dominated the market, and because graphics were memory hungry 'nice to haves', people worked predominantly in a text user interface. If you wanted graphics, you had to have an expensive, powerful PC and most people didn't think that was really worth the cost.

Just think of it now - we use Microsoft Word (or of course LibreOffice!) in a graphical manner, even though we are writing text, there are fonts, buttons, icons and ribbon menus, it's all graphical. Back in the 1980s, a word processor was pure plain text! The primary operating system, MS-DOS was also text based.

Tux tip!

The Terminal is where you enter command line instructions. The Terminal can also be referred to as a console, prompt, CLI, command line or shell.

Step forward a bit to when Linux came out, around 1991. Anyone who wanted to use their PC in a quick, efficient manner with a Microsoft machine would use Windows 3.0 to run their graphical programs like Aldus PageMaker or Lotus 1-2-3, but they would often open an MS-DOS prompt. This meant they could type a few commands. They would do that so that they could quickly rename a file or move it from one place to another. Sure, even then users could use a graphical file explorer, select the file(s) they wanted to rename or move and drag them to the relevant location, but a lot of people were finding that DOS was still quicker to use if you knew the commands. If you spent a few days learning the rudimentary commands of DOS, you could really use your PC in a quick, no nonsense way. Coupled with the graphical usefulness of Windows, you could cut workload down by a good deal. Even today, when using modern GUI O/S's like Windows and Mac OS, I still find myself going to Start going to Start>Run and typing in `cmd` (Windows) or Terminal.app (mac) to access the command prompt to do something quickly.

Over the years, the people who use the Desktop operating systems such as Mac and Windows have mainly forgotten about the fact that the command prompt still exists, but for systems people, the people that want to do operations in bulk, or do them quickly, they still use it often. In Windows, Microsoft created a thing called PowerShell for this exact reason. MacOS X brought about the bourne shell (the UNIX shell from BSD) and of course, Linux gives us the Bash shell (the Bourne Again SHell, which is just an enhanced edition of the bourne shell that you find on a mac).

Once you get to grips with it, it's not scary at all. Also, if you want to have a career in Linux, you'll find that you will need to know around the command line.

Why do we still use the command line today?

Imagine the following scenario:

You have 300 files in a folder (call the folder 'work'), and within that folder you have 2 other folders, one called 'old' and one called 'new'. Your task is to separate the files in work to the new and old folder. You must put the files that are older than 1 month into old, and the files less than 1 month into new. In a graphical file manager, you would need to right click each of the 300 files separately to find out their creation date, and then move each of the files one by one into their relevant folder. This operation is seriously time-consuming.

How's about the command-line option: type in one line at the shell, and the files are automatically sorted into each folder, determined by today's date. Sure, you would know that you would have to type something like

`'find /usr -ctime +30 -exec ls -ld {} \;'` to get it to happen, but hey, it's still faster than going through 300 files.

There are yet many other reasons that you may want to have a CLI, for example, a User Administrator has 3 dead (crashed) programs on 3 different workstations across a building. He has two choices: Go run around the building like a headless chicken and kill the programs manually at each workstation, or sit at his or her desk and SSH into each machine, using the CLI, killing each of the programs, finishing the job 20 times faster.

The joy of text

If you start to use Linux a lot, you'll start to find it's a lot easier just to issue a direct command instead of clicking on a bunch of different icons to do something you could do in one command at the CLI. In fact, even Microsoft have, as of 2017 stated that they 'Love Linux'. They've brought the Bash shell via Ubuntu to the Windows desktop, and MacOS has the bash shell built into it since the beginning of MacOS X. The reason for this? Because it's where people get serious about being effective system administrators and programmers. If you fancy a career in being a system administrator then you should read the bottom of this article.

How do I access the Terminal?

The Terminal application can be run by pressing Ctrl+Alt+T (Ubuntu and similar) or clicking the icon in your Applications > Accessories menu, or pressing the Windows key to enter the apps dash and typing ***terminal***, then press return.

There are many terminal apps available and have slightly different features, but effectively they all do the same thing. Common terminal apps are called Gnome Terminal, Konsole, rxvt, xterm, terminator and lxterminal to name but a few.

Some simple commands you might recognise

GNU/Linux has a lot of commands, luckily though, you'll probably only ever want to know around 10-15 of them for anything at the CLI if you simply want to remain a desktop user. You can find the files for these commands in folders like **/bin** and **/usr/bin** on your Linux box.

If you have used computers since at least the late 90s, then you will probably remember DOS, you'll find that the syntax of DOS commands compared to UNIX commands are often similar, and even some of the commands themselves are the same. Have a look at this comparison table if you can remember any DOS commands:

DOS/Windows Command	UNIX Counter-part	What it does
dir	ls	Shows the files in the current folder
cls	clear	Clears the screen
type	cat	Shows the contents of a file, eg cat foo.txt
move	mv	Move a file from one place to another, eg: mv foo /home/
copy	cp	Copies a file, eg: cp /home/foo /users/foo
ren / rename	mv	The mv (move) command also doubles up for renaming
echo	echo	output (or echo) something to the screen

Some more commands, plus how to get help on almost any command

Here are a few more advanced commands, along with their explanations and one example for each:

Command	Action	Example
chmod	Changes permissions for files	<code>chmod u=rwx myfile.txt</code>
chown	change the owner for a file or folder	<code>chown myuser /home/myfile.txt</code>
chgrp	change the group owner for a file or folder	<code>chgrp mygroup /home/myfile.txt</code>
<code>mv</code>	Moves or renames a file	<code>mv /home/me/myfile.txt /home/them/</code>
<code>grep</code>	searches text for results	<code>cat myfile.txt grep "hello"</code>
<code>ls</code>	list the files in a folder	<code>ls -l</code>
<code>find</code>	finds files or file designators	<code>find /home -name "myfile.txt"</code>
<code>mount</code>	mounts a drive/ block device for use	<code>mount /dev/cdrom /mnt/cdrom</code>
<code>ps</code>	Show the list of live processes	<code>ps ax more</code>
<code>kill</code>	kill a process (running software) by process ID	<code>kill -9 12345</code>
<code>tar</code>	de facto Unix archiver - archives files into one file (a .tar file). Using the 'z' flag uses compression (gzip).	<code>tar zxvf myarchive.tar.gz</code>
<code>rpm</code>	The Red Hat Package manager, used to install, remove, upgrade and query RPM packages.	<code>rpm -Uvh mynewpackage.i386.rpm</code>

If you ever need to know how to use a command, or just find out what a command does **man** (short for manual) is your friend. For example, by typing `man ls`, a comprehensive guide is given on using the `ls` command, what its purpose is and what its syntax is. For more information on using `man`, type **man man**.

There is not a better source for quick, easy to find information on almost every Linux command. The only times when `man` lets you down, is when you don't know that a command that does a specific purpose exists, thus, you don't know the name of the command, so you don't know what manual page you want to ask `man` for. If `man` doesn't fit the bill, you can get by with reading the rest of The Ultimate Linux Newbie Guide and some googling!

Now Learn More!

- Get yourself started with the fundamentals of Linux System Administration on the command-line with the free [Linux 101 presentation deck here at the Ultimate Linux Newbie Guide](#)
- Read my article on [how I got a career in Linux](#).
- [Sign up to the Linux Foundation for official certification](#). Training courses are at your time and pace, remotely managed and are internationally recognised.

Some other helpful links:

- [ViM Text Editor 101](#)
- [Linux Security 101](#)
- [Sysadmin Articles at the ULNG](#)